Ciclo de Treinamento, Particionamento e Modelagem Prática

Aula 04 — Repasse Prático: Divisão Treino/Teste, o Loop de Otimização e Diferenciação entre Classificação e Regressão

Instrutor: **Prof. Allan Spadini**

Redes Neurais Profundas | Graduação Faculdade Infnet

A Regra de Ouro do Particionamento

Isolamento estatístico entre Treino e Teste para eliminação total de Data Leakage

ETAPA 1
Dataset Bruto Completo

ETAPA 2
Particionamento Cego (Split 80/20)

ETAPA 3
Ajuste do Scaler Estritamente no Treino

ETAPA 4
Aplicação Transparente no Teste

Treino (100%) Teste Cego (0%)

X_train, y_train (100%)

INGESTÃO INICIAL **1. Dataset Bruto Completo**

Todo o conjunto de dados carregado na memória (ex: 1.338 linhas de seguro saúde ou 350 clientes). Não toque em médias ou desvios ainda!

Operação PyTorch/Scikit: `Nenhum (aguardando split)`

REGRA DE OURO INVIOLÁVEL

NUNCA chame scaler.fit() ou fit_transform() no dataset inteiro ou no conjunto de teste. O teste DEVE ser cego e simular os dados de amanhã que o modelo nunca viu.

◀ Anterior Próximo ▶

O Ciclo Canônico de Treinamento no PyTorch

As 5 etapas fundamentais executadas estritamente em cada mini-batch

1. **Forward Pass**
2. Cálculo da Função de Perda
3. Zerar Gradientes Antigos
4. Backward Pass (Autograd)
5. Atualização dos Pesos

PASSO 1 DE 5

Forward Pass

```
y_pred = model(X_batch)
```
PyTorch

Os tensores do mini-batch (X) atravessam as camadas lineares e funções de ativação. A rede produz previsões brutas (logits ou valores contínuos).

O que acontece internamente: Calcula ativações e mantém o grafo de operações dinâmico na memória RAM/GPU.

O LOOP SAGRADO DE TREINAMENTO (POR ÉPOCA)

```
# 1. Modo de Treino
model.train()

# 2. Iteração pelos Mini-batches
for X_batch, y_batch in train_loader:
  1. y_pred = model(X_batch)
  2. loss = criterion(y_pred, y_batch)
  3. optimizer.zero_grad()
  4. loss.backward()
  5. optimizer.step()
```

◀ Passo Anterior | Próximo Passo ▶

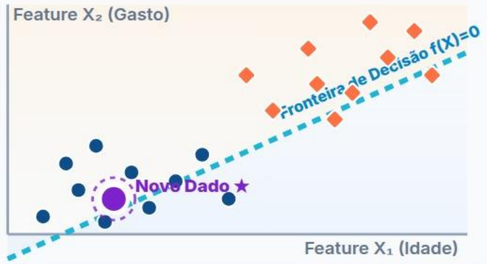
Classificação vs. Regressão no PyTorch

Fronteiras de decisão discretas vs. curvas contínuas de ajuste e resíduos

Visão Comparativa (Gráficos Lado a Lado) | Foco: Classificação (Fronteira de Decisão) | Foco: Regressão (Ajuste de Curva Contínua)

OBJETIVO: SEPARAR REGIÕES (DISCRETO)

Classificação: Fronteira de Decisão



👆 *Clique no gráfico para mover o ponto* **Predição: Classe 1 (Comprador / Alta Satisfação)**

Camada Saída: `nn.Linear(dim, C)`

Ativação: Nenhuma (Logits brutos)

Função de Perda: `CrossEntropyLoss`

Métricas: Acurácia, F1, Matriz Confusão

OBJETIVO: ESTIMAR QUANTIDADE (CONTÍNUO)

Regressão: Curva Contínua & Resíduos



Idade: **43 anos** — **Custo Previsto: R$ 5.251,00**

Camada Saída: `nn.Linear(dim, 1)`

Ativação: Nenhuma (Linear direta)

Função de Perda: `nn.MSELoss()`

Métricas: MAE, RMSE, R^2 (Variância)

Modo Treino vs. Modo Avaliação

O impacto de model.train(), model.eval() e with torch.no_grad()

Modo Treinamento: model.train() | Modo Avaliação: model.eval() + torch.no_grad()

GRAFO COMPUTACIONAL

Autograd & Memória

Ativo por padrão. Cada operação guarda ativações intermediárias na RAM/VRAM para viabilizar o cálculo posterior de derivadas parciais no loss.backward().

```
torch.is_grad_enabled() == True
```

REGULARIZAÇÃO DROPOUT

Comportamento de Neurônios

Desativa neurônios estocasticamente com probabilidade p (ex: 20% a 50%) em cada batch para evitar coadaptação e combater overfitting.

```
nn.Dropout(p): Ativo (aleatório)
```

BATCH NORMALIZATION

Estatísticas de Normalização

Normaliza usando a média e variância DO MINI-BATCH atual, além de atualizar as médias móveis globais (running mean e running var).

```
Usa média/var do batch
```

Roteiro de Repasse dos Notebooks da Aula

Estrutura dos 3 ambientes práticos que executaremos juntos hoje

PRÁTICA 1 • AQUECIMENTO
`pratica_primeira_rede_iris`

PRÁTICA 2 • TRANSIÇÃO DIRETA
`aula_04_pipelines_dados_simples`

PRÁTICA 3 • PIPELINE PROFISSIONAL
`aula_04_pipelines_dados_classificacao_regressao`

PRÁTICA 1 • AQUECIMENTO

📁 `pratica_primeira_rede_iris.ipynb`

Construção passo a passo de um classificador para o dataset Iris com gabarito

Dataset: Iris (Scikit-Learn)

Foco: Classificação em 3 classes (Setosa, Versicolor, Virginica)

- Dataset simples e canônico de 150 flores e 4 atributos numéricos.
- Exercício guiado com lacunas para preencher: forward pass, criterion e optimizer.
- Demonstração visual do porquê 1 única camada linear não separa classes não-lineares.
- Acompanha o arquivo prática_primeira_rede_iris_gabarito.ipynb com solução completa.

Dica do Professor: Abra o notebook correspondente agora para acompanhar e executar cada célula em tempo real!

Síntese da Aula & Próximos Passos

Consolidação do pipeline básico e conexão com a estabilidade de treinamento na Aula 05

CONSOLIDAÇÃO DE CONCEITOS

Síntese da Aula 04

- **Split antes do Scaler:** O conjunto de teste é cego. Nunca permita que médias ou desvios de teste vazem para o treino.
- **As 5 etapas canônicas:** Forward → Loss → zero_grad() → Backward → step() formam o coração de todo treino em PyTorch.
- **Cabeças e Funções de Perda:** CrossEntropyLoss (logits) para classificação vs MSELoss (saída linear 1D) para regressão.
- **train() vs eval():** Desative gradientes e configure camadas de regularização para uma inferência justa e eficiente.

🎯 **Conexão com o Projeto 1:** Estes pipelines compõem a estrutura básica que você utilizará para entregar a primeira avaliação prática da disciplina.

PRÓXIMA PARADA • AULA 05

Estabilidade de Treinamento

Agora que dominamos o ciclo de treino e os dados, como garantimos que redes neurais mais profundas aprendam sem que os gradientes explodam ou desapareçam?

1. Inicializações de Pesos: Xavier Uniform vs Kaiming He e por que a inicialização errada trava o modelo na época 0.

2. Normalizações Modernas: BatchNorm1d vs LayerNorm mantendo as ativações em faixas estáveis.

3. Regularização Ativa: Dropout e Learning Rate Schedulers combatendo overfitting em campo.

→ **Nos vemos na Aula 05:** Estabilidade, Inicialização & Regularização!